

Amazon SDE1

Coding Interview Guide

27 problems from your playlist. One page each. Brute force code AND optimal code, side by side.

A note on video transcripts: this session's network policy blocks youtube.com, so no transcripts could be pulled from the linked playlist. Every explanation below is written from general algorithmic knowledge, not transcribed from any video — see the per-problem note under each title. To get real transcripts pulled in, enable broader network access (or allowlist youtube.com) in this environment's settings and re-run.

Table of Contents

#	Problem	LC	Pattern
1	Two Sum	#1	Hash Map
2	Contains Duplicate	#217	Hash Set
3	Group Anagrams	#49	Hash Map
4	Product of Array Except Self	#238	Prefix/Suffix Products
5	Subarray Sum Equals K	#560	Prefix Sum + Hash Map
6	Valid Palindrome	#125	Two Pointers
7	Container With Most Water	#11	Two Pointers
8	Longest Substring Without Repeating Characters	#3	Sliding Window
9	Minimum Window Substring	#76	Sliding Window
10	Valid Parentheses	#20	Stack
11	Design Min Stack	#155	Stack (Auxiliary Min Stack)
12	Daily Temperatures	#739	Monotonic Stack
13	Evaluate Reverse Polish Notation	#150	Stack
14	Implement Queue using Stacks	#232	Stack (Two-Stack Trick)
15	Implement Stack using Queues	#225	Queue (Rotation Trick)
16	Maximum Depth of Binary Tree	#104	DFS / Recursion
17	Binary Tree Level Order Traversal	#102	BFS
18	Binary Tree Vertical Order Traversal	#314	BFS + Hash Map (Column Index)
19	Clone Graph	#133	DFS/BFS + Hash Map
20	Course Schedule	#207	Graph DFS / Topological Sort (Cycle Detection)
21	Binary Search	#704	Binary Search
22	Find Minimum in Rotated Sorted Array	#153	Binary Search (Modified)
23	LRU Cache	#146	Hash Map + Doubly Linked List
24	Random Pick with Weight	#528	Prefix Sum + Binary Search
25	Rotting Oranges	#994	Multi-Source BFS
26	Maximum Subarray	#53	Dynamic Programming (Kadane's Algorithm)
27	Top K Frequent Elements	#347	Bucket Sort (or Heap)

Two Sum

LeetCode #1 | **EASY** | Pattern: Hash Map | Video: "Two Sum - Leetcode 1 - HashMap - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given an array of integers and a target, return the indices of the two numbers that add up to the target. Exactly one solution exists; can't use the same element twice. *Ask:* Exactly one solution guaranteed? Can't reuse the same element twice?

2. Brute force: Check every pair of indices and test if they sum to target.

```
def two_sum(nums, target):
    n = len(nums)
    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] + nums[j] == target:
                return [i, j]
    return []
```

3. Complexity: $O(n^2)$ time, $O(1)$ space

4. Bottleneck: Checking every pair re-scans the array for a partner I could just remember instead.

5. Optimize: Store each value's index as you scan; before adding, check if target minus current value was already seen — one pass instead of two nested loops.

6. Code it (narrate as you write):

```
def two_sum(nums, target):
    seen = {} # value -> index
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

9. Final complexity: $O(n)$ time, $O(n)$ space

7. Test: nums=[2,7,11,15], target=9 -> map builds {2:0}, then 7's complement 2 is already in the map -> [0,1]. Edge: nums=[3,3], target=6 -> works since we check the map before inserting.

8. Other edge cases: no valid pair exists; negative numbers; target achieved by first two elements.

Memory hook: "I need the complement — remember what I've already seen."

Contains Duplicate

LeetCode #217 | **EASY** | Pattern: Hash Set | Video: "Contains Duplicate - Leetcode 217 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given an array of integers, return true if any value appears at least twice, and false if every element is distinct. Ask: Can the array be empty or have one element? Any bound on value range?

2. Brute force: Compare every pair of elements for equality.

```
def contains_duplicate(nums):
    n = len(nums)
    for i in range(n):
        for j in range(i + 1, n):
            if nums[i] == nums[j]:
                return True
    return False
```

3. Complexity: $O(n^2)$ time, $O(1)$ space

4. Bottleneck: Comparing every pair re-checks values I could just remember in a set as I go.

5. Optimize: A hash set lets me check 'have I seen this before' in $O(1)$ average time, so one pass suffices.

6. Code it (narrate as you write):

```
def contains_duplicate(nums):
    seen = set()
    for num in nums:
        if num in seen:
            return True
        seen.add(num)
    return False

# One-liner alt: len(set(nums)) != len(nums)
```

9. Final complexity: $O(n)$ time, $O(n)$ space

7. Test: nums=[1,2,3,1] -> seen builds {1,2,3}, hits 1 again -> True. Edge: empty array -> loop never runs, False.

8. Other edge cases: all unique; all identical; single element.

Memory hook: "Seen it before? Set tells you instantly."

Group Anagrams

LeetCode #49 | **MEDIUM** | Pattern: Hash Map | Video: "Group Anagrams - Categorize Strings by Count - Leetcode 49" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given an array of strings, group the anagrams together (any order). Two strings are anagrams if they contain the same letters with the same counts. Ask: Case-sensitive? Only lowercase letters guaranteed, or could there be other characters?

2. Brute force: Compare every string against every other by sorting both and checking equality.

```
def group_anagrams(strs):
    groups = []
    used = [False] * len(strs)
    for i in range(len(strs)):
        if used[i]:
            continue
        group = [strs[i]]
        for j in range(i + 1, len(strs)):
            if not used[j] and sorted(strs[i]) == sorted(strs[j]):
                group.append(strs[j])
                used[j] = True
        groups.append(group)
    return groups
```

3. Complexity: $O(n^2 * k \log k)$ time, $O(n * k)$ space

4. Bottleneck: Sorting each string to compare it against every other string is repeated, wasted work.

5. Optimize: A 26-count tuple (or sorted string) is identical for all anagrams of a word, so it works as a hash map key that groups them in one pass.

6. Code it (narrate as you write):

```
from collections import defaultdict

def group_anagrams(strs):
    groups = defaultdict(list)
    for word in strs:
        count = [0] * 26
        for ch in word:
            count[ord(ch) - ord('a')] += 1
        groups[tuple(count)].append(word)
    return list(groups.values())
```

9. Final complexity: $O(n * k)$ time, $O(n * k)$ space

7. Test: ['eat','tea','tan'] -> 'eat'/'tea' share a count key, 'tan' gets its own. Edge: empty string -> its own group.

8. Other edge cases: single string in the list; all strings anagrams of each other; duplicate identical strings.

Memory hook: "Same letter counts = same key = same bucket."

Product of Array Except Self

LeetCode #238 | **MEDIUM** | Pattern: Prefix/Suffix Products | Video: "Product of Array Except Self - Leetcode 238 - Python" — NeetCode.

Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given an array, return an array where each element is the product of all other elements, without using division, in $O(n)$. Ask: Can the array contain zero(s)? Division truly disallowed?

2. Brute force: For each index, loop over the rest of the array and multiply everything else.

```
def product_except_self(nums):
    n = len(nums)
    result = []
    for i in range(n):
        product = 1
        for j in range(n):
            if j != i:
                product *= nums[j]
        result.append(product)
    return result
```

3. Complexity: $O(n^2)$ time, $O(1)$ extra space

4. Bottleneck: Re-multiplying the whole array minus one element, for every element, redoes overlapping work.

5. Optimize: Answer at i = (product of everything left of i) * (product of everything right of i). Build prefix products left-to-right, then fold in suffix products right-to-left.

6. Code it (narrate as you write):

```
def product_except_self(nums):
    n = len(nums)
    result = [1] * n
    prefix = 1
    for i in range(n):
        result[i] = prefix
        prefix *= nums[i]
    suffix = 1
    for i in range(n - 1, -1, -1):
        result[i] *= suffix
        suffix *= nums[i]
    return result
```

9. Final complexity: $O(n)$ time, $O(1)$ extra space

7. Test: [1,2,3,4] -> prefix pass gives [1,1,2,6], suffix pass folds in to [24,12,8,6]. Edge: one zero, e.g. [1,2,0,4] -> [0,0,8,0].

8. Other edge cases: two or more zeros (all outputs 0); negative numbers; array of length 2.

Memory hook: "Everything to my left, times everything to my right."

Subarray Sum Equals K

LeetCode #560 | **MEDIUM** | Pattern: Prefix Sum + Hash Map | Video: "Subarray Sum Equals K - Prefix Sums - Leetcode 560 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given an array of integers and an integer k, return the total number of contiguous subarrays whose sum equals k. Numbers can be negative. Ask: Can values be negative? Does an empty subarray count (no, subarrays are non-empty contiguous runs)?

2. Brute force: Check every subarray's sum directly (or extend a running sum per start index).

```
def subarray_sum(nums, k):
    count = 0
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            if total == k:
                count += 1
    return count
```

3. Complexity: $O(n^2)$ time, $O(1)$ space

4. Bottleneck: Recomputing each subarray's sum from scratch re-adds numbers I've already summed before.

5. Optimize: $\text{sum}(i..j) = \text{prefix}[j] - \text{prefix}[i-1]$. So $\text{sum}(i..j) == k$ means $\text{prefix}[i-1] == \text{prefix}[j] - k$. Track how many times each prefix sum has occurred so far.

6. Code it (narrate as you write):

```
from collections import defaultdict

def subarray_sum(nums, k):
    count = 0
    prefix_sum = 0
    seen = defaultdict(int)
    seen[0] = 1 # empty prefix
    for num in nums:
        prefix_sum += num
        count += seen[prefix_sum - k]
        seen[prefix_sum] += 1
    return count
```

9. Final complexity: $O(n)$ time, $O(n)$ space

7. Test: [1,1,1], k=2 -> prefix sums 1,2,3; seen[0]=1 upfront; count becomes 2. Edge: k=0 with a zero in the array needs seen[0]=1 handled correctly.

8. Other edge cases: all negative numbers; k not achievable at all; single-element array equal to k.

Memory hook: "Two equal prefix sums — the gap between them sums to k."

Valid Palindrome

LeetCode #125 | **EASY** | Pattern: Two Pointers | Video: "Valid Palindrome - Leetcode 125 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given a string, return true if it's a palindrome after converting to lowercase and removing all non-alphanumeric characters. *Ask:* Only alphanumeric characters count, case-insensitive? Can the string be empty?

2. Brute force: Build a cleaned version of the string, then compare it to its reverse.

```
def is_palindrome(s):
    cleaned = [c.lower() for c in s if c.isalnum()]
    return cleaned == cleaned[::-1]
```

3. Complexity: $O(n)$ time, $O(n)$ space

4. Bottleneck: Building a whole cleaned copy of the string uses $O(n)$ extra space I don't need.

5. Optimize: Use two pointers from both ends, skipping non-alphanumeric characters in place, comparing directly — no extra string built.

6. Code it (narrate as you write):

```
def is_palindrome(s):
    left, right = 0, len(s) - 1
    while left < right:
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1
        if s[left].lower() != s[right].lower():
            return False
        left += 1
        right -= 1
    return True
```

9. Final complexity: $O(n)$ time, $O(1)$ space

7. Test: "A man, a plan, a canal: Panama" -> pointers skip punctuation/case, converge to True. Edge: string of only punctuation -> pointers cross immediately, True.

8. Other edge cases: single character; numbers mixed with letters; already-lowercase palindrome.

Memory hook: "Skip junk, compare from both ends inward."

Container With Most Water

LeetCode #11 | **MEDIUM** | Pattern: Two Pointers | Video: "Container with Most Water - Leetcode 11 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given heights of vertical lines, find two lines that together with the x-axis form a container holding the most water. Return the max area. Ask: Can heights be zero? At least two lines guaranteed?

2. Brute force: Check every pair of lines and compute the area they'd contain.

```
def max_area(height):
    best = 0
    n = len(height)
    for i in range(n):
        for j in range(i + 1, n):
            area = (j - i) * min(height[i], height[j])
            best = max(best, area)
    return best
```

3. Complexity: $O(n^2)$ time, $O(1)$ space

4. Bottleneck: Checking every pair of lines recomputes areas for pairs that can't possibly beat the current best.

5. Optimize: Start with the widest container (both ends). The shorter wall is always the bottleneck, so move it inward — moving the taller wall could never help.

6. Code it (narrate as you write):

```
def max_area(height):
    left, right = 0, len(height) - 1
    best = 0
    while left < right:
        h = min(height[left], height[right])
        best = max(best, (right - left) * h)
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1
    return best
```

9. Final complexity: $O(n)$ time, $O(1)$ space

7. Test: [1,8,6,2,5,4,8,3,7] -> pointers start at ends (area $7*1=7$), move shorter side inward, converge to $best=49$. Edge: all equal heights -> widest pair (the ends) is optimal immediately.

8. Other edge cases: two lines only; one very tall line surrounded by short ones; strictly increasing heights.

Memory hook: "Move the shorter wall — the taller one was never the limit."

Longest Substring Without Repeating Characters

LeetCode #3 | **MEDIUM** | Pattern: Sliding Window | Video: "Longest Substring Without Repeating Characters - Leetcode 3 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given a string, find the length of the longest substring without repeating characters. Ask: Empty string possible? Only ASCII, or could there be unicode?

2. Brute force: Check every substring for uniqueness directly.

```
def length_of_longest_substring(s):
    best = 0
    for i in range(len(s)):
        seen = set()
        for j in range(i, len(s)):
            if s[j] in seen:
                break
            seen.add(s[j])
            best = max(best, j - i + 1)
    return best
```

3. Complexity: $O(n^2)$ time, $O(\min(n, \text{charset}))$ space

4. Bottleneck: Restarting a fresh scan from every index re-examines characters already confirmed unique.

5. Optimize: Expand the window right; if the new character is already in the window, shrink from the left until the duplicate is gone — window never revisits a position twice.

6. Code it (narrate as you write):

```
def length_of_longest_substring(s):
    seen = set()
    left = 0
    best = 0
    for right in range(len(s)):
        while s[right] in seen:
            seen.remove(s[left])
            left += 1
        seen.add(s[right])
        best = max(best, right - left + 1)
    return best
```

9. Final complexity: $O(n)$ time, $O(\min(n, \text{charset}))$ space

7. Test: "abcabcbb" -> window grows to "abc" (len 3), hits repeat 'a', shrinks left, best stays 3. Edge: "" -> loop never runs, best=0.

8. Other edge cases: all identical characters ("aaaa"); string with no repeats at all; single character.

Memory hook: "Duplicate found? Shrink from the left until it's gone."

Minimum Window Substring

LeetCode #76 | **HARD** | Pattern: Sliding Window | Video: "Minimum Window Substring - Airbnb Interview Question - Leetcode 76" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given strings s and t , find the minimum window substring of s that contains every character of t (with at least its multiplicity), or empty string if none exists. Ask: Multiplicity of characters in t matters (need at least that many of each)? What if t is empty or longer than s ?

2. Brute force: Check every substring of s and test whether it contains all of t 's characters.

```
from collections import Counter

def min_window(s, t):
    need = Counter(t)
    best = ""
    for i in range(len(s)):
        for j in range(i, len(s)):
            window = Counter(s[i:j+1])
            if all(window[c] >= need[c] for c in need):
                if best == "" or j - i + 1 < len(best):
                    best = s[i:j+1]
    return best
```

3. Complexity: $O(n^3)$ time, $O(n)$ space

4. Bottleneck: Checking every substring directly recomputes its character counts from scratch each time.

5. Optimize: Expand right until the window satisfies all of t 's counts; then shrink from the left as much as possible while still valid, recording the smallest valid window found.

6. Code it (narrate as you write):

```
from collections import Counter

def min_window(s, t):
    if not t:
        return ""
    need = Counter(t)
    missing = len(t)
    left = 0
    best = (float('inf'), 0, 0)
    for right, ch in enumerate(s):
        if need[ch] > 0:
            missing -= 1
            need[ch] -= 1
        while missing == 0:
            if right - left + 1 < best[0]:
                best = (right - left + 1, left, right)
            need[s[left]] += 1
            if need[s[left]] > 0:
                missing += 1
            left += 1
    return "" if best[0] == float('inf') else s[best[1]:best[2]+1]
```

9. Final complexity: $O(n + m)$ time, $O(\text{charset})$ space

7. Test: $s = \text{"ADOBECODEBANC"} , t = \text{"ABC"} \rightarrow$ window expands/shrinks to "BANC". Edge: t empty $\rightarrow$ return "" immediately.

8. Other edge cases: t longer than s (no answer, return ""); t 's characters not present in s at all; s equals t exactly.

Memory hook: "Expand until valid, then shrink until it breaks — record the smallest valid window."

Valid Parentheses

LeetCode #20 | **EASY** | Pattern: Stack | Video: "Valid Parentheses - Stack - Leetcode 20 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given a string of just '{}[]()' characters, determine if the brackets are validly matched and nested. Ask: Only the six bracket characters appear, no other text mixed in?

2. Brute force: Repeatedly remove any adjacent matched pair like '()', '[]', '{}'. '[]' from the string until no more removals are possible; valid iff the string becomes empty.

```
def is_valid(s):
    pairs = ["()", "[]", "{}"]
    changed = True
    while changed:
        changed = False
        for p in pairs:
            if p in s:
                s = s.replace(p, "", 1)
                changed = True
    return len(s) == 0
```

3. Complexity: $O(n^2)$ time, $O(n)$ space

4. Bottleneck: Repeatedly scanning and removing matched pairs re-scans the whole string over and over.

5. Optimize: Push opening brackets. On a closing bracket, it must match the top of the stack (the most recently opened, still-unclosed bracket); if not, invalid.

6. Code it (narrate as you write):

```
def is_valid(s):
    stack = []
    pairs = {'(': ')', '[': ']', '{': '}'}
    for ch in s:
        if ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
        else:
            stack.append(ch)
    return not stack
```

9. Final complexity: $O(n)$ time, $O(n)$ space

7. Test: "{}[]()" -> every open matches the most recent close, stack empties, True. Edge: "()" -> first char is a close with an empty stack, False.

8. Other edge cases: single unmatched bracket; only opening brackets; only closing brackets; empty string (trivially valid).

Memory hook: "Push opens; a close must match the most recent open."

Design Min Stack

LeetCode #155 | **MEDIUM** | Pattern: Stack (Auxiliary Min Stack) | Video: "Design Min Stack - Amazon Interview Question - Leetcode 155 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Design a stack supporting push, pop, top, and getMin, all in $O(1)$ time. Ask: Do push/pop/top/getMin ALL need true $O(1)$, not just amortized? Duplicate values possible?

2. Brute force: Plain stack; getMin scans the whole stack every time it's called.

```
class MinStack:
    def __init__(self):
        self.stack = []
    def push(self, val):
        self.stack.append(val)
    def pop(self):
        self.stack.pop()
    def top(self):
        return self.stack[-1]
    def get_min(self):
        return min(self.stack)  #  $O(n)$  !
```

3. Complexity: push/pop/top $O(1)$, getMin $O(n)$ time, $O(n)$ space

4. Bottleneck: Scanning the whole stack on every getMin call redoes a search I could track incrementally instead.

5. Optimize: Maintain a second stack that always stores the running minimum alongside each push, so it's popped in lockstep and always instantly available.

6. Code it (narrate as you write):

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []
    def push(self, val):
        self.stack.append(val)
        m = val if not self.min_stack else min(val, self.min_stack[-1])
        self.min_stack.append(m)
    def pop(self):
        self.stack.pop()
        self.min_stack.pop()
    def top(self):
        return self.stack[-1]
    def get_min(self):
        return self.min_stack[-1]
```

9. Final complexity: $O(1)$ all ops time, $O(n)$ space

7. Test: push(-2), push(0), push(-3), getMin()->-3, pop(), top()->0, getMin()->-2. Edge: push same min twice then pop once -> getMin still correct via $\leq$ comparison.

8. Other edge cases: popping down to empty; strictly increasing pushes; strictly decreasing pushes.

Memory hook: "A second stack, always remembering the min-so-far."

Daily Temperatures

LeetCode #739 | **MEDIUM** | Pattern: Monotonic Stack | Video: "Daily Temperatures - Monotonic Stack - Leetcode 739 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given daily temperatures, return an array where `answer[i]` is how many days until a warmer temperature; 0 if none exists. Ask: Return 0 when no warmer day exists? Temperatures guaranteed positive integers?

2. Brute force: For each day, scan forward until a warmer day is found.

```
def daily_temperatures(temps):
    n = len(temps)
    answer = [0] * n
    for i in range(n):
        for j in range(i + 1, n):
            if temps[j] > temps[i]:
                answer[i] = j - i
                break
    return answer
```

3. Complexity: $O(n^2)$ time, $O(1)$ extra space

4. Bottleneck: Scanning forward from every day re-examines days already ruled out as 'not warmer' for a previous day.

5. Optimize: Keep a stack of day-indices whose warmer day hasn't been found yet. When a warmer day appears, it resolves every cooler day still waiting on the stack.

6. Code it (narrate as you write):

```
def daily_temperatures(temps):
    answer = [0] * len(temps)
    stack = [] # indices, decreasing temps
    for i, temp in enumerate(temps):
        while stack and temps[stack[-1]] < temp:
            j = stack.pop()
            answer[j] = i - j
        stack.append(i)
    return answer
```

9. Final complexity: $O(n)$ time, $O(n)$ space

7. Test: [73,74,75,71,69,72,76,73] -> stack resolves each day as a warmer one appears; day 0 waits 1 day. Edge: strictly decreasing temps -> nothing ever resolves, all zeros.

8. Other edge cases: all identical temperatures (never strictly warmer, all zeros); single day; strictly increasing temps.

Memory hook: "Stack of unresolved days; a warmer day resolves everyone smaller beneath it."

Evaluate Reverse Polish Notation

LeetCode #150 | **MEDIUM** | Pattern: Stack | Video: "Evaluate Reverse Polish Notation - Leetcode 150 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given tokens of a postfix arithmetic expression (+ - * /), evaluate it and return the integer result. Ask: Division truncates toward zero, not floor? Expression always guaranteed valid?

2. Brute force: There's no simpler alternative to a stack here; a naive idea (convert to infix and use a general parser) is more complex, not simpler.

```
# A general infix-parser approach still needs precedence
# handling and is strictly more complex than the stack
# approach below for postfix notation - not shown as a
# separate 'slower but simpler' baseline; go straight to
# the stack-based evaluation, which is already optimal.
```

3. Complexity: N/A time, N/A space

4. Bottleneck: N/A for this one -- postfix is meant to be evaluated with a stack directly; there's no meaningfully slower correct alternative.

5. Optimize: Postfix means an operator always combines the two MOST RECENT operands — a stack naturally holds 'most recent' in order.

6. Code it (narrate as you write):

```
def eval_rpn(tokens):
    stack = []
    ops = {'+', '-', '*', '/'}
    for tok in tokens:
        if tok not in ops:
            stack.append(int(tok))
        else:
            b = stack.pop()
            a = stack.pop()
            if tok == '+': stack.append(a + b)
            elif tok == '-': stack.append(a - b)
            elif tok == '*': stack.append(a * b)
            else: stack.append(int(a / b)) # truncate toward 0
    return stack.pop()
```

9. Final complexity: O(n) time, O(n) space

7. Test: ["2","1","+","3","*"] -> push 2,1; '+' -> 3; push 3; '*' -> 9. Edge: negative division like -7/2 needs int(a/b), not a//b.

8. Other edge cases: single-number expression, no operators; division resulting in an exact integer; deeply nested operator chains.

Memory hook: "Numbers pile up; an operator pops the top two and pushes the result."

Implement Queue using Stacks

LeetCode #232 | **EASY** | Pattern: Stack (Two-Stack Trick) | Video: "Implement Queue using Stacks - Leetcode 232 - Python" — NeetCodeIO.
Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Implement a FIFO queue using only two stacks, supporting push, pop, peek, and empty. Ask: Are push/pop/peek/empty all that's required? Any constraint on amortized vs. worst-case time?

2. Brute force: Use one stack for input; to pop/peek (need the OLDEST item), pop everything into a temp list, take the item, then push everything back — done every single call.

```
class MyQueue:
    def __init__(self):
        self.stack = []
    def push(self, x):
        self.stack.append(x)
    def pop(self):
        temp = []
        while len(self.stack) > 1:
            temp.append(self.stack.pop())
        result = self.stack.pop()
        while temp:
            self.stack.append(temp.pop())
        return result # O(n) every call!
```

3. Complexity: pop/peek $O(n)$ every call time, $O(n)$ space

4. Bottleneck: Reversing the whole stack on every single pop/peek call redoes the same shuffling repeatedly.

5. Optimize: Two stacks: 'in_stack' for pushes, 'out_stack' for pops. Only transfer in->out when out_stack is empty; each element is moved at most once, ever.

6. Code it (narrate as you write):

```
class MyQueue:
    def __init__(self):
        self.in_stack = []
        self.out_stack = []
    def push(self, x):
        self.in_stack.append(x)
    def _shift(self):
        if not self.out_stack:
            while self.in_stack:
                self.out_stack.append(self.in_stack.pop())
    def pop(self):
        self._shift()
        return self.out_stack.pop()
    def peek(self):
        self._shift()
        return self.out_stack[-1]
    def empty(self):
        return not self.in_stack and not self.out_stack
```

9. Final complexity: $O(1)$ amortized time, $O(n)$ space

7. Test: push(1), push(2), peek()->1, pop()->1, empty()->False. Edge: interleaved push/pop -- out_stack only refills once fully drained, preserving order.

8. Other edge cases: popping until truly empty; a single element; many pushes with no pops in between.

Memory hook: "Two stacks: reverse once into 'out', drain it fully before refilling."

Implement Stack using Queues

LeetCode #225 | **EASY** | Pattern: Queue (Rotation Trick) | Video: "Implement Stack using Queues - Leetcode 225 - Python" — NeetCode.
Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Implement a LIFO stack using only queues, supporting push, pop, top, and empty. Ask: Are push/pop/top/empty all required? Push allowed to be $O(n)$ if pop/top stay $O(1)$?

2. Brute force: Use two queues; to push, add to the empty one, then move everything from the other queue behind it, then swap names — $O(n)$ per push done via an extra queue.

```
from collections import deque

class MyStack:
    def __init__(self):
        self.q1 = deque()
        self.q2 = deque()
    def push(self, x):
        self.q2.append(x)
        while self.q1:
            self.q2.append(self.q1.popleft())
        self.q1, self.q2 = self.q2, self.q1
    def pop(self):
        return self.q1.popleft()
    def top(self):
        return self.q1[0]
    def empty(self):
        return not self.q1
```

3. Complexity: push $O(n)$ time, $O(n)$ space

4. Bottleneck: Using two full queues and swapping them every push does more bookkeeping than necessary.

5. Optimize: Single queue: after appending the new element, rotate the queue by rotating everything BEFORE it to behind it — the new element ends up at the front, acting like the top of a stack.

6. Code it (narrate as you write):

```
from collections import deque

class MyStack:
    def __init__(self):
        self.q = deque()
    def push(self, x):
        self.q.append(x)
        for _ in range(len(self.q) - 1):
            self.q.append(self.q.popleft())
    def pop(self):
        return self.q.popleft()
    def top(self):
        return self.q[0]
    def empty(self):
        return not self.q
```

9. Final complexity: push $O(n)$, pop/top $O(1)$ time, $O(n)$ space

7. Test: push(1), push(2), top() → 2, pop() → 2. Edge: single-element stack -- rotation loop runs zero times, element stays at front.

8. Other edge cases: popping to empty; pushing then immediately popping repeatedly; large sequential pushes.

Memory hook: "Rotate the queue so the newest push lands at the front."

Maximum Depth of Binary Tree

LeetCode #104 | **EASY** | Pattern: DFS / Recursion | Video: "Maximum Depth of Binary Tree - 3 Solutions - Leetcode 104 - Python" —

NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given the root of a binary tree, return its maximum depth (number of nodes on the longest root-to-leaf path). Ask: Depth counted in nodes (not edges)? Empty tree has depth 0?

2. Brute force: There isn't a distinct slower approach here; the natural solution IS the recursive one below. An alternative worth showing is iterative BFS (level counting).

```
from collections import deque

def max_depth_bfs(root):
    if not root:
        return 0
    depth = 0
    queue = deque([root])
    while queue:
        depth += 1
        for _ in range(len(queue)):
            node = queue.popleft()
            if node.left: queue.append(node.left)
            if node.right: queue.append(node.right)
    return depth
```

3. Complexity: O(n) time, O(n) worst case space

7. Test: root=3,left=9,right=20(15,7) -> left depth 1, right depth 2, total 1+max(1,2)=3. Edge: empty tree (root=None) -> 0.

8. Other edge cases: single-node tree (depth 1); completely skewed tree (depth = node count); perfectly balanced tree.

Memory hook: "1 plus whichever child subtree goes deeper."

4. Bottleneck: N/A here -- the recursive definition IS the efficient solution; only alternative is iterative BFS, not a slower one.

5. Optimize: Depth of a tree = 1 + max(depth of left subtree, depth of right subtree); null node has depth 0. Direct recursive definition.

6. Code it (narrate as you write):

```
def max_depth(root):
    if root is None:
        return 0
    left = max_depth(root.left)
    right = max_depth(root.right)
    return 1 + max(left, right)
```

9. Final complexity: O(n) time, O(h) space

Binary Tree Level Order Traversal

LeetCode #102 | **MEDIUM** | Pattern: BFS | Video: "Binary Tree Level Order Traversal - BFS - Leetcode 102" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given the root of a binary tree, return the node values grouped level by level (left to right), as a list of lists. Ask: Return values grouped by level, left to right within each level? Empty tree returns []?

2. Brute force: DFS could work but requires tracking depth explicitly and appending to the right list — more error-prone than BFS's natural level grouping.

```
def level_order_dfs(root):
    result = []
    def dfs(node, depth):
        if not node:
            return
        if depth == len(result):
            result.append([])
        result[depth].append(node.val)
        dfs(node.left, depth + 1)
        dfs(node.right, depth + 1)
    dfs(root, 0)
    return result
```

3. Complexity: O(n) time, O(n) space

4. Bottleneck: DFS with explicit depth tracking works but is more error-prone than BFS's natural level grouping.

5. Optimize: BFS naturally processes one level at a time. Snapshot the queue's current size before processing, so that batch is exactly one level's worth of nodes.

6. Code it (narrate as you write):

```
from collections import deque

def level_order(root):
    if not root:
        return []
    result = []
    queue = deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left: queue.append(node.left)
            if node.right: queue.append(node.right)
        result.append(level)
    return result
```

9. Final complexity: O(n) time, O(n) space

7. Test: root=3,left=9,right=20(15,7) -> [[3],[9,20],[15,7]]. Edge: empty tree -> [] immediately.

8. Other edge cases: single-node tree; only left children (skewed); only right children (skewed).

Memory hook: "Snapshot queue size = process exactly one level."

Binary Tree Vertical Order Traversal

LeetCode #314 | **MEDIUM** | Pattern: BFS + Hash Map (Column Index) | Video: "Binary Tree Vertical Order Traversal - Leetcode 314 - Python" — NeetCodeIO. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given a binary tree, return the node values grouped by vertical column, ordered left to right; within a column, top to bottom, and left-to-right for ties at the same node depth. *Ask:* Ties at the same column AND same depth ordered left to right? Column indices can be negative?

2. Brute force: DFS while tracking (column, row) pairs, then sort all (column, row, value) triples at the end.

```
def vertical_order_dfs(root):
    triples = []
    def dfs(node, col, row):
        if not node:
            return
        triples.append((col, row, node.val))
        dfs(node.left, col - 1, row + 1)
        dfs(node.right, col + 1, row + 1)
    dfs(root, 0, 0)
    triples.sort() # by col, then row, then value
    result = {}
    for col, row, val in triples:
        result.setdefault(col, []).append(val)
    return [result[c] for c in sorted(result)]
```

3. Complexity: $O(n \log n)$ time, $O(n)$ space

4. Bottleneck: A DFS-then-sort-everything approach does more sorting than needed when BFS already visits top-to-bottom, left-to-right.

5. Optimize: BFS processes nodes level by level (top to bottom, left to right) automatically, so within a column, values are already appended in correct order — no final sort needed, just group by column index and sort column KEYS at the end.

6. Code it (narrate as you write):

```
from collections import deque, defaultdict

def vertical_order(root):
    if not root:
        return []
    columns = defaultdict(list)
    queue = deque([(root, 0)])
    while queue:
        node, col = queue.popleft()
        columns[col].append(node.val)
        if node.left: queue.append((node.left, col - 1))
        if node.right: queue.append((node.right, col + 1))
    return [columns[c] for c in sorted(columns)]
```

9. Final complexity: $O(n \log n)$ time, $O(n)$ space

7. Test: root=3, left=9, right=8 (left=4) -> columns -1:[9,4], 0:[3,8]? trace carefully per structure. Edge: two nodes same column+level from different parents -- BFS order still resolves left-to-right correctly.

8. Other edge cases: empty tree (return []); single node (one column); all nodes in one column (a straight line).

Memory hook: "BFS order is already top-to-bottom, left-to-right — just bucket by column."

Clone Graph

LeetCode #133 | **MEDIUM** | Pattern: DFS/BFS + Hash Map | Video: "Clone Graph - Depth First Search - Leetcode 133" — NeetCode.

Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given a reference node in a connected undirected graph, return a deep copy (clone) of the entire graph. Ask: Graph is connected and undirected? Guaranteed no self-loops beyond what's explicit in neighbors lists?

2. Brute force: There isn't a meaningfully 'slower' correct alternative — you fundamentally need some way to avoid re-cloning (and infinitely looping on) a node you've already visited. Naive recursion WITHOUT a visited map would infinite-loop on any cycle.

```
# Recursion without tracking visited nodes:
def clone_graph_broken(node):
    if not node:
        return None
    copy = Node(node.val)
    for neighbor in node.neighbors:
        copy.neighbors.append(clone_graph_broken(neighbor))
    return copy
# BROKEN: infinite recursion on any cycle back to a
# node already being cloned (graphs here are typically cycl
```

3. Complexity: Infinite loop on cycles time, N/A space

4. Bottleneck: Recursing without tracking visited nodes infinite-loops on any cycle, which this graph type has.

5. Optimize: A hash map from original node to its clone both (a) prevents infinite recursion on cycles and (b) lets neighbors correctly reference already-created clones.

6. Code it (narrate as you write):

```
def clone_graph(node):
    if not node:
        return None
    old_to_new = {}
    def dfs(n):
        if n in old_to_new:
            return old_to_new[n]
        copy = Node(n.val)
        old_to_new[n] = copy
        for neighbor in n.neighbors:
            copy.neighbors.append(dfs(neighbor))
        return copy
    return dfs(node)
```

9. Final complexity: $O(V + E)$ time, $O(V)$ space

7. Test: node 1 with neighbors [2,4] -> clone recurses, map prevents re-cloning shared neighbors. Edge: single node with itself as a neighbor -- map entry created before recursing avoids infinite loop.

8. Other edge cases: single node with no neighbors; a simple cycle of 3 nodes; disconnected part (not expected, but worth asking).

Memory hook: "Map original -> clone FIRST, then recurse into neighbors."

Course Schedule

LeetCode #207 | **MEDIUM** | Pattern: Graph DFS / Topological Sort (Cycle Detection) | Video: "Course Schedule - Graph Adjacency List - Leetcode 207" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given numCourses and prerequisite pairs [a, b] (must take b before a), return whether it's possible to finish all courses (i.e., the prerequisite graph has no cycle). Ask: Prerequisites given as [a,b] meaning b before a? Can there be duplicate prerequisite pairs?

2. Brute force: Try every possible course order (permutations) and check if any order satisfies all prerequisites — combinatorially expensive.

```
from itertools import permutations

def can_finish_brute(numCourses, prerequisites):
    for order in permutations(range(numCourses)):
        position = {c: i for i, c in enumerate(order)}
        if all(position[b] < position[a] for a, b in prerequisites):
            return True
    return False # O(n!) - only feasible for tiny n
```

3. Complexity: $O(n!)$ time, $O(n)$ space

4. Bottleneck: Trying every possible course ordering directly is combinatorially expensive (factorial).

5. Optimize: A valid course order exists iff the prerequisite graph has no cycle. DFS each course, marking nodes 'in current path'; if DFS revisits a node still in the current path, that's a cycle.

6. Code it (narrate as you write):

```
def can_finish(numCourses, prerequisites):
    graph = [[] for _ in range(numCourses)]
    for a, b in prerequisites:
        graph[a].append(b)
    state = [0] * numCourses # 0=unvisited, 1=visiting, 2=done
    def dfs(course):
        if state[course] == 1:
            return False # cycle!
        if state[course] == 2:
            return True
        state[course] = 1
        for pre in graph[course]:
            if not dfs(pre):
                return False
        state[course] = 2
        return True
    return all(dfs(c) for c in range(numCourses))
```

9. Final complexity: $O(V + E)$ time, $O(V + E)$ space

7. Test: numCourses=2, prereqs=[[1,0]] -> course 0 has no deps, course 1 depends on 0, no cycle, True. Edge: [[0,0]] -- course depends on itself, DFS revisits itself mid-path, cycle detected, False.

8. Other edge cases: no prerequisites at all (always True); a long chain with no cycle; disconnected groups of courses.

Memory hook: "Mark 'in-progress'; revisiting an in-progress node means a cycle."

Binary Search

LeetCode #704 | **EASY** | Pattern: Binary Search | Video: "Binary Search - Leetcode 704 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given a sorted array of integers and a target, return its index, or -1 if not present, in $O(\log n)$. Ask: Array guaranteed sorted ascending? Duplicate values possible (does it matter which index is returned)?

2. Brute force: Scan every element left to right until the target is found.

```
def search(nums, target):
    for i, num in enumerate(nums):
        if num == target:
            return i
    return -1
```

3. Complexity: $O(n)$ time, $O(1)$ space

4. Bottleneck: Scanning left to right ignores the sorted order that lets me eliminate half the array per comparison.

5. Optimize: Sorted order means comparing the middle element tells you which entire half to discard — halving the search space every step.

6. Code it (narrate as you write):

```
def search(nums, target):
    left, right = 0, len(nums) - 1
    while left <= right:
        mid = (left + right) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1
```

9. Final complexity: $O(\log n)$ time, $O(1)$ space

7. Test: [-1,0,3,5,9,12], target=9 -> mid narrows down, found at index 4. Edge: target not present, e.g. target=2 -> left crosses right, return -1.

8. Other edge cases: single-element array; target is the first or last element; empty array.

Memory hook: "Middle tells you which whole half to throw away."

Find Minimum in Rotated Sorted Array

LeetCode #153 | **MEDIUM** | Pattern: Binary Search (Modified) | Video: "Find Minimum in Rotated Sorted Array - Binary Search - Leetcode 153 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given a rotated sorted array with all-unique values, find the minimum element in $O(\log n)$. Ask: All values unique guaranteed? Could the array be not rotated at all (already sorted)?

2. Brute force: Scan the whole array to find the minimum directly.

```
def find_min(nums):  
    return min(nums) # O(n), ignores the sorted structure
```

3. Complexity: $O(n)$ time, $O(1)$ space

4. Bottleneck: Scanning every element to find the min ignores the sorted-then-rotated structure that binary search can exploit.

5. Optimize: Compare $\text{nums}[\text{mid}]$ to $\text{nums}[\text{right}]$. If $\text{nums}[\text{mid}] > \text{nums}[\text{right}]$, the minimum is in the right half (rotation point is there); otherwise it's in the left half (mid could BE the minimum, so don't exclude it).

6. Code it (narrate as you write):

```
def find_min(nums):  
    left, right = 0, len(nums) - 1  
    while left < right:  
        mid = (left + right) // 2  
        if nums[mid] > nums[right]:  
            left = mid + 1  
        else:  
            right = mid  
    return nums[left]
```

9. Final complexity: $O(\log n)$ time, $O(1)$ space

7. Test: [3,4,5,1,2] -> $\text{mid}=5 > \text{right}(2)$ so search right half; converges to index 3, value 1. Edge: not rotated, e.g. [1,2,3,4,5] -> $\text{nums}[\text{mid}]$ never exceeds $\text{nums}[\text{right}]$, converges to index 0.

8. Other edge cases: rotation point at the very start (no rotation); rotation point at the very end; two-element array.

Memory hook: "Compare mid to the right edge — that tells you which side hides the rotation."

LRU Cache

LeetCode #146 | **MEDIUM** | Pattern: Hash Map + Doubly Linked List | Video: "LRU Cache - Twitch Interview Question - Leetcode 146" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Design a Least Recently Used (LRU) cache with a fixed capacity, supporting get(key) and put(key, value), both in O(1) average time. Ask: Capacity guaranteed at least 1? Does 'use' include both get AND put (both refresh recency)?

2. Brute force: Use a plain dict for values plus a separate list tracking usage order; every get/put requires an O(n) scan or shift of that order list.

```
class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.data = {}
        self.order = [] # least-recent at index 0
    def get(self, key):
        if key not in self.data:
            return -1
        self.order.remove(key) # O(n)
        self.order.append(key)
        return self.data[key]
    def put(self, key, value):
        if key in self.data:
            self.order.remove(key) # O(n)
        elif len(self.data) >= self.cap:
            oldest = self.order.pop(0) # O(n)
            del self.data[oldest]
        self.data[key] = value
        self.order.append(key)
```

3. Complexity: O(n) per get/put time, O(n) space

4. Bottleneck: Using a plain list to track recency order means removing/reordering an item costs O(n) (shifting).

5. Optimize: A hash map gives O(1) lookup by key; a doubly linked list gives O(1) removal/insertion at any position (no shifting) to track recency order, moving a node to the front on every access.

6. Code it (narrate as you write):

```
class Node:
    def __init__(self, k, v):
        self.key, self.val = k, v
        self.prev = self.next = None

class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.map = {}
        self.left = Node(0, 0) # least recent side
        self.right = Node(0, 0) # most recent side
        self.left.next = self.right
        self.right.prev = self.left
    def _remove(self, node):
        node.prev.next, node.next.prev = node.next, node.prev
    def _insert(self, node):
        node.prev, node.next = self.right.prev, self.right
        self.right.prev.next = node
        self.right.prev = node
    def get(self, key):
        if key not in self.map:
            return -1
        self._remove(self.map[key]); self._insert(self.map[key])
        return self.map[key].val
    def put(self, key, value):
        if key in self.map:
            self._remove(self.map[key])
        node = Node(key, value)
        self.map[key] = node
        self._insert(node)
        if len(self.map) > self.cap:
            lru = self.left.next
            self._remove(lru); del self.map[lru.key]
```

9. Final complexity: O(1) get/put time, O(capacity) space

7. Test: capacity=2: put(1,1),put(2,2),get(1)->1,put(3,3) evicts key 2,get(2)->-1. Edge: updating an existing key's value must move it to most-recent, not just overwrite in place.

8. Other edge cases: capacity of 1 (every put evicts the previous key); getting a key right after inserting it; repeated puts to the same key.

Memory hook: "Map for O(1) lookup, doubly linked list for O(1) reorder — evict from the far end."

Random Pick with Weight

LeetCode #528 | **MEDIUM** | Pattern: Prefix Sum + Binary Search | Video: "Random Pick with Weight | Bucket approach | Leetcode #528 | Binary search" — Techdose. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given an array of positive weights, design a structure that picks an index at random, where the probability of picking index i is proportional to $\text{weights}[i]$. Ask: Weights guaranteed positive integers? Does `pick_index` get called many times (yes, so precompute once)?

2. Brute force: Build a 'bucket' array where each index appears $\text{weights}[i]$ times, then pick a uniformly random element from that array directly.

```
import random

class Solution:
    def __init__(self, w):
        self.buckets = []
        for i, weight in enumerate(w):
            self.buckets.extend([i] * weight) # O(total weights)
    def pick_index(self):
        return random.choice(self.buckets)
```

3. Complexity: build $O(\sum w)$, pick $O(1)$ time, $O(\sum w)$ space

4. Bottleneck: Building a bucket array of size $\sum(\text{weights})$ wastes huge memory when weights are large.

5. Optimize: Build a prefix-sum array of weights (cumulative ranges). Pick a random number in $[1, \text{total}]$, then binary search for the first prefix sum $\geq$ that number — that index's range contains the random point.

6. Code it (narrate as you write):

```
import random, bisect

class Solution:
    def __init__(self, w):
        self.prefix = []
        total = 0
        for weight in w:
            total += weight
            self.prefix.append(total)
        self.total = total
    def pick_index(self):
        target = random.randint(1, self.total)
        return bisect.bisect_left(self.prefix, target)
```

9. Final complexity: build $O(n)$, pick $O(\log n)$ time, $O(n)$ space

7. Test: $w=[1,3] \rightarrow \text{prefix}=[1,4]$, $\text{total}=4$; random in $[1,4]$: 1 $\rightarrow$ index 0, 2..4 $\rightarrow$ index 1 (3x more likely, matching weight). Edge: single weight of 1 $\rightarrow$ always index 0.

8. Other edge cases: all weights equal (uniform pick); one very large weight dominating; two weights only.

Memory hook: "Weights become ranges (prefix sums); binary search finds which range the dart landed in."

Rotting Oranges

LeetCode #994 | **MEDIUM** | Pattern: Multi-Source BFS | Video: "Rotting Oranges - Leetcode 994 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: A grid has empty cells, fresh oranges, and rotten oranges. Every minute, rot spreads to adjacent fresh oranges. Return the minutes until no fresh orange remains, or -1 if impossible. Ask: 4-directional spread only, not diagonal? Return 0 if no fresh oranges exist at the start?

2. Brute force: Simulate minute by minute: rescan the WHOLE grid every minute to find newly-infected oranges.

```
def oranges_rotting_brute(grid):
    rows, cols = len(grid), len(grid[0])
    minutes = 0
    while True:
        fresh_left = False
        to_rot = []
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] == 1:
                    fresh_left = True
                    for dr, dc in [(-1,0), (1,0), (0,-1), (0,1)]:
                        nr, nc = r+dr, c+dc
                        if 0<=nr<rows and 0<=nc<cols and grid[nr][nc] == 0:
                            to_rot.append((r,c))
                            break
        if not to_rot:
            return -1 if fresh_left else minutes
        for r, c in to_rot:
            grid[r][c] = 2
        minutes += 1
```

3. Complexity: $O((rows*cols)^2)$ time, $O(rows*cols)$ space

4. Bottleneck: Rescanning the entire grid every single minute to find newly-infected oranges redoes work already known.

5. Optimize: Seed a BFS queue with EVERY rotten orange at once. Each full BFS level = exactly one minute of simultaneous spreading; no rescanning needed.

6. Code it (narrate as you write):

```
from collections import deque

def oranges_rotting(grid):
    rows, cols = len(grid), len(grid[0])
    queue = deque()
    fresh = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2: queue.append((r, c))
            elif grid[r][c] == 1: fresh += 1
    minutes = 0
    dirs = [(-1,0), (1,0), (0,-1), (0,1)]
    while queue and fresh > 0:
        minutes += 1
        for _ in range(len(queue)):
            r, c = queue.popleft()
            for dr, dc in dirs:
                nr, nc = r+dr, c+dc
                if 0<=nr<rows and 0<=nc<cols and grid[nr][nc]==1:
                    grid[nr][nc] = 2
                    fresh -= 1
                    queue.append((nr, nc))
    return minutes if fresh == 0 else -1
```

9. Final complexity: $O(rows * cols)$ time, $O(rows * cols)$ space

7. Test: $[[2,1,1],[1,1,0],[0,1,1]] \rightarrow$ rot spreads outward minute by minute, takes 4 minutes total. Edge: a fresh orange sealed off by walls with no path to any rotten orange $\rightarrow$ fresh never hits 0, return -1.

8. Other edge cases: no fresh oranges at all (return 0); no rotten oranges but fresh ones exist (return -1); grid that's entirely empty cells.

Memory hook: "Seed BFS with every rotten orange at once — BFS levels ARE the minutes."

Maximum Subarray

LeetCode #53 | **MEDIUM** | Pattern: Dynamic Programming (Kadane's Algorithm) | Video: "Maximum Subarray - Amazon Coding Interview Question - Leetcode 53 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given an integer array (possibly with negatives), find the contiguous subarray with the largest sum and return that sum. Ask: Array guaranteed non-empty? Must return a real subarray sum (can't return 0 for an empty selection)?

2. Brute force: Check every possible subarray's sum directly (or extend a running sum per start index).

```
def max_subarray(nums):
    best = nums[0]
    for i in range(len(nums)):
        total = 0
        for j in range(i, len(nums)):
            total += nums[j]
            best = max(best, total)
    return best
```

3. Complexity: $O(n^2)$ time, $O(1)$ space

4. Bottleneck: Recomputing every subarray's sum from scratch redoes overlapping additions across nearby subarrays.

5. Optimize: If the running sum ever goes negative, it can only hurt any future subarray — drop it and restart at the current element instead of carrying it forward.

6. Code it (narrate as you write):

```
def max_subarray(nums):
    current = best = nums[0]
    for num in nums[1:]:
        current = max(num, current + num)
        best = max(best, current)
    return best
```

9. Final complexity: $O(n)$ time, $O(1)$ space

7. Test: [-2,1,-3,4,-1,2,1,-5,4] -> running sum resets after negatives, builds up to best=6 via [4,-1,2,1]. Edge: all-negative array, e.g. [-3,-1,-2] -- starting at nums[0] (not 0) correctly gives -1.

8. Other edge cases: single-element array; entire array is the answer (all positive); array with a single large positive surrounded by negatives.

Memory hook: "A negative running sum is dead weight — drop it and start fresh."

Top K Frequent Elements

LeetCode #347 | **MEDIUM** | Pattern: Bucket Sort (or Heap) | Video: "Top K Frequent Elements - Bucket Sort - Leetcode 347 - Python" — NeetCode. Transcript not retrievable in this session (YouTube is blocked by this environment's network policy); explained from general algorithmic knowledge, not the video.

1. Restate & clarify: Given an integer array and an integer k, return the k most frequently occurring elements, in any order. Ask: Order of the output doesn't matter? k guaranteed $\leq$ number of distinct elements?

2. Brute force: Count frequencies, then fully sort all distinct values by count and take the top k.

```
from collections import Counter

def top_k_frequent(nums, k):
    counts = Counter(nums)
    sorted_items = sorted(counts.items(), key=lambda x: -x[1])
    return [val for val, cnt in sorted_items[:k]]
```

3. Complexity: $O(n \log n)$ time, $O(n)$ space

4. Bottleneck: Fully sorting all distinct values by frequency does more ordering work than needed when I only want the top k.

5. Optimize: Frequency can never exceed n, so make 'buckets' indexed by frequency (0..n); put each value in its frequency's bucket, then read off values from the highest bucket down until k are collected — no comparison sort needed.

6. Code it (narrate as you write):

```
from collections import Counter

def top_k_frequent(nums, k):
    counts = Counter(nums)
    n = len(nums)
    buckets = [[] for _ in range(n + 1)]
    for val, freq in counts.items():
        buckets[freq].append(val)
    result = []
    for freq in range(n, 0, -1):
        for val in buckets[freq]:
            result.append(val)
            if len(result) == k:
                return result
    return result
```

9. Final complexity: $O(n)$ time, $O(n)$ space

7. Test: [1,1,1,2,2,3], k=2 -> counts {1:3,2:2,3:1}; buckets collect from freq=3 down, stop once 2 collected -> [1,2]. Edge: k equals number of distinct elements -> loop collects everything before returning.

8. Other edge cases: all elements with the same frequency; k=1 (single most frequent); many low-frequency singletons.

Memory hook: "Bucket index = frequency — no sorting needed, frequency is bounded by n."

Pattern Cheat Sheet

Interview clue	Pattern
Need a pair/complement fast	Hash Map (Two Sum, Contains Duplicate)
Same letters/counts grouped	Hash Map canonical key (Group Anagrams)
Product/sum except self, no division	Prefix * Suffix products
# subarrays summing to k, negatives allowed	Prefix Sum + Hash Map
Palindrome check, O(1) space	Two Pointers from both ends
Maximize area/container between two points	Two Pointers, move the shorter side
Longest substring with a constraint	Sliding Window
Matching brackets / most-recent-first	Stack
Next greater/warmer element to the right	Monotonic Stack
Design O(1) getMin alongside a stack	Auxiliary min-stack
Implement stack with queues (or vice versa)	Two-structure rotation trick
Tree depth / recursive tree property	DFS Recursion
Process a tree/grid level by level	BFS with queue-size snapshot
Deep copy a graph/structure with cycles	DFS/BFS + Hash Map (orig -> clone)
Can all tasks finish given dependencies	Cycle detection / Topological Sort
Sorted array + search	Binary Search
Rotated sorted array	Binary Search vs. right boundary
O(1) get/put + evict least-recently-used	Hash Map + Doubly Linked List
Pick randomly with weighted probability	Prefix Sum + Binary Search
Multiple sources spreading simultaneously	Multi-Source BFS
Max sum of a contiguous subarray	Kadane's Algorithm (DP)
Top K frequent, bound on frequency	Bucket Sort by frequency

Sources

Primary source

The playlist at youtube.com/playlist?list=PL_6hP3Z1GQqYrrF1fETaEpkLAX6M8E7j was provided. This session's network policy blocks youtube.com, so none of its videos could be fetched or transcribed. No quotes in this guide are attributed to, or copied from, any video transcript.

Supporting sources

LeetCode problem statements (for exact constraints/return formats) and general, widely-known algorithmic technique names and explanations, written independently.

Videos named in the provided playlist (titles/channels, not transcribed)

- "Two Sum - Leetcode 1 - HashMap - Python" — NeetCode
- "Contains Duplicate - Leetcode 217 - Python" — NeetCode
- "Group Anagrams - Categorize Strings by Count - Leetcode 49" — NeetCode
- "Product of Array Except Self - Leetcode 238 - Python" — NeetCode
- "Subarray Sum Equals K - Prefix Sums - Leetcode 560 - Python" — NeetCode
- "Valid Palindrome - Leetcode 125 - Python" — NeetCode
- "Container with Most Water - Leetcode 11 - Python" — NeetCode
- "Longest Substring Without Repeating Characters - Leetcode 3 - Python" — NeetCode
- "Minimum Window Substring - Airbnb Interview Question - Leetcode 76" — NeetCode
- "Valid Parentheses - Stack - Leetcode 20 - Python" — NeetCode
- "Design Min Stack - Amazon Interview Question - Leetcode 155 - Python" — NeetCode
- "Daily Temperatures - Monotonic Stack - Leetcode 739 - Python" — NeetCode
- "Evaluate Reverse Polish Notation - Leetcode 150 - Python" — NeetCode
- "Implement Queue using Stacks - Leetcode 232 - Python" — NeetCodeIO
- "Implement Stack using Queues - Leetcode 225 - Python" — NeetCode
- "Maximum Depth of Binary Tree - 3 Solutions - Leetcode 104 - Python" — NeetCode
- "Binary Tree Level Order Traversal - BFS - Leetcode 102" — NeetCode
- "Binary Tree Vertical Order Traversal - Leetcode 314 - Python" — NeetCodeIO
- "Clone Graph - Depth First Search - Leetcode 133" — NeetCode
- "Course Schedule - Graph Adjacency List - Leetcode 207" — NeetCode
- "Binary Search - Leetcode 704 - Python" — NeetCode
- "Find Minimum in Rotated Sorted Array - Binary Search - Leetcode 153 - Python" — NeetCode
- "LRU Cache - Twitch Interview Question - Leetcode 146" — NeetCode
- "Random Pick with Weight | Bucket approach | Leetcode #528 | Binary search" — Techdose
- "Rotting Oranges - Leetcode 994 - Python" — NeetCode
- "Maximum Subarray - Amazon Coding Interview Question - Leetcode 53 - Python" — NeetCode
- "Top K Frequent Elements - Bucket Sort - Leetcode 347 - Python" — NeetCode